Du cadrage au paquet : construire et deployer une application intélligente complète

Développement et déploiement d'applications intelligentes

TP1 à TP6, élèves ingénieurs

1 Cadrage 2 Modèle 3 API 4 Tests 5 Interface 6 Paquet

A. Larhlimi

Six étapes pour passer d'une idée à une application livrée

Plan

1 Cadrer — TP1

Choisir l'approche, fixer les critères

2 Entraîner — TP2

Un modèle mesuré sur des avis jamais vus

3 Exposer — TP3

Une API qui refuse les entrées invalides

4 Tester — TP4

Chercher ce qui casse avant l'utilisateur

5 Montrer — TP5

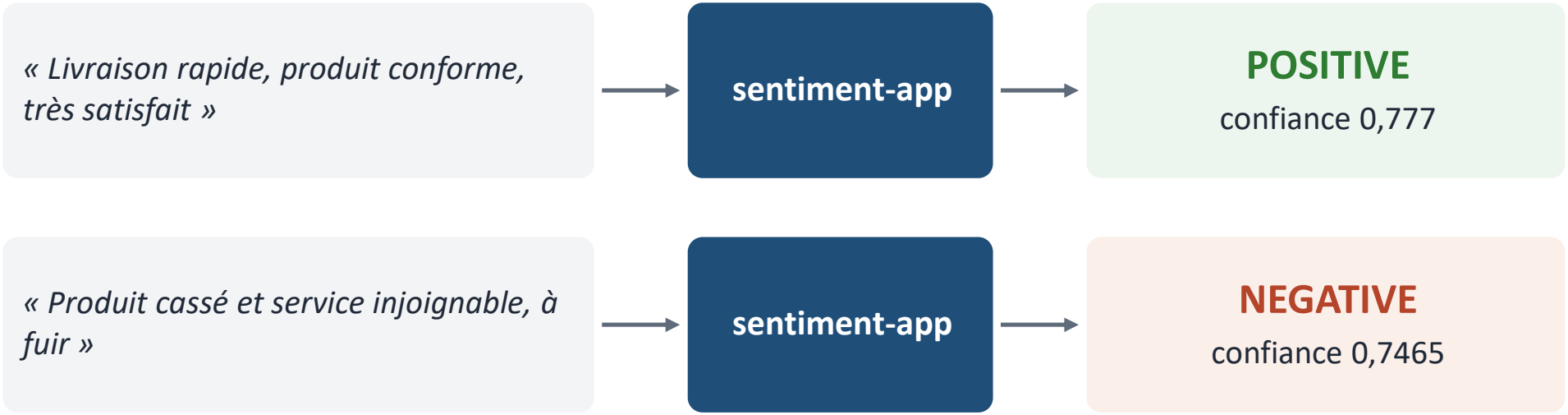
Une interface qui ne parle qu'à l'API

6 Livrer — TP6

Un paquet installable partout

Un seul projet du TP1 au TP6 : sentiment-app

TP1-6



Des avis clients en français, deux classes : positive et negative. Le même dépôt grandit de TP en TP.

Quatre couches, chacune ne parle qu'à sa voisine

Archi

Paquet installable (TP6) : la roue sentiment_app

Interface (TP5)
ui/app.py, ui/client.py

HTTP

API (TP3)
api/main.py, schemas.py, model.py

appel Python

Modèle (TP2)
sentiment_app/predict.py, sentiment.joblib

Tests (TP4)
pytest, hypothesis, ruff, mypy et la CI vérifient les trois couches

Partie 1, TP1

Cadrer avant de coder

1

Un projet d'apprentissage se définit par une tâche, une expérience et une mesure

TP1

T Tâche

Classer un avis : positive ou negative

E Expérience

232 avis étiquetés, le jeu d'entraînement

P Mesure

Le F1 macro sur 58 avis jamais vus

D'après Tom Mitchell (1997) : un programme apprend si sa mesure P sur la tâche T progresse avec l'expérience E.

Quatre critères suffisent pour choisir une approche

TP1

Critère	Règles métier	Apprentissage automatique	RAG
(a) Données étiquetées	Inutiles	Indispensables, en quantité	Inutiles : des documents
(b) Vérifiabilité	Totale : on cite la règle	Partielle : on explique les poids	Bonne, si la réponse cite ses sources
(c) Coût par requête	Quasi nul	Faible	Élevé : un modèle de langage
(d) Conséquence d'une erreur	Maîtrisée	Mesurable, à encadrer	À contrôler : hallucinations

Le bon choix est l'approche la plus simple qui satisfait les critères d'acceptation.

Trois cas, trois approches différentes

TP1

Pharmacie : alerter sur un dosage dangereux	Règlement intérieur : répondre aux salariés	Service client : repérer les avis négatifs
↓	↓	↓
Règles métier, validées par le pharmacien	**RAG : des réponses citées**	**Apprentissage automatique**
(b) chaque alerte se justifie ; (d) une erreur peut être grave	(a) pas d'étiquettes, des documents ; (b) on relit la source	(a) des avis étiquetés existent ; (c) coût par avis faible

Une fiche de cadrage tient en huit rubriques

En détail TP1

Rubrique	Pour le cas 3 : repérer les avis négatifs
Besoin en une phrase	Repérer chaque jour les avis négatifs, pour y répondre en priorité
Utilisateur final	L'équipe du service client
Approche retenue	Une seule case cochée : apprentissage automatique
Justification	(a) des avis étiquetés existent ; (c) le coût par avis est faible
Données nécessaires	Des avis en français, étiquetés positif ou négatif
Métrique de succès	F1 macro d'au moins 0,85 ; rappel des négatifs suivi
Conséquence d'une erreur	Un mécontent manqué coûte plus qu'une fausse alerte
Risques éthiques	Données personnelles dans les avis ; un humain tranche les cas douteux

Les tests exigent 20 caractères rédigés par rubrique, une seule case cochée et deux critères cités.

Les critères d'acceptation se fixent avant de construire

TP1

Critère d'acceptation	Vérifié par
F1 macro d'au moins 0,85 sur un jeu de test jamais vu	`test_f1_au_dessus_du_seuil (TP2)`
Toute entrée invalide refusée par un 422, jamais par un 500	`test_limites.py (TP4)`
Aucune erreur franche non déclarée sur le jeu d'or	`test_jeu_or.py (TP4)`
Sous une confiance de 0,60, l'API s'abstient	`api/model.py (TP4)`

Fixés après coup, les critères décrivent seulement ce que le modèle sait déjà faire.

38 tests vérifient le TP1 : l'environnement, les fiches, l'historique

En détail TP1

23

`test_environment.py`

Python 3.11, environnement activé, paquets, dossiers et fichiers, dépendances épinglées, .gitignore, aucun .env commité

12

`test_cadrage.py`

Trois fiches, huit rubriques présentes et remplies, une approche cochée, deux critères cités

3

`test_git_history.py`

Au moins un commit, deux auteurs distincts, .venv jamais commité

```
python -m pytest tests/test_environment.py tests/test_cadrage.py tests/test_git_history.py -q
```

Attendu en fin de TP1 : 38 passed.

À retenir : le cadrage décide de tout le reste

1. Nommer la tâche, l'expérience et la mesure
2. Choisir l'approche avec (a) à (d) : la plus simple qui suffit
3. Fixer les critères avant de construire

Les contrôles du TP

T1
3 failed voulus, 31 passed, 4 skipped

↓

T2
34 passed, 4 skipped

↓

T7
38 passed sur un clone neuf

Partie 2, TP2

Entraîner et mesurer le modèle

2

Le modèle en deux étapes : des mots aux nombres, puis des nombres au verdict

TP2

Un avis

« Taille trop petite et retour payant au déballage, sauf que ensuite produit conforme… »

→

1. Vectoriser : TF-IDF

316 termes, un nombre par terme

TF : fréquent dans l'avis

IDF : rare dans les autres avis

→

2. Classer : régression logistique

Un poids par terme, une somme : z

La sigmoïde fait de z une probabilité

Ici z = −0,1343 : negative, 0,5335

Les deux étapes forment un seul objet, le Pipeline, sauvegardé d'un bloc : la même transformation à l'entraînement et en production.

L'essentiel est la méthode : quatre règles pour un résultat fiable

TP2

1 Découper avant de vectoriser

58 avis mis de côté avant d'apprendre le vocabulaire : sinon, le test fuit

2 Mesurer chaque classe

negative : précision 0,844, rappel 0,931 ; F1 macro 0,879

3 Garder la trace des mesures

metrics.json dans Git : la CI vérifie le F1 sans le modèle

4 Comprendre les erreurs

7 erreurs, toutes des avis qui se retournent : l'ordre des mots est ignoré

Le même modèle donne 0,879 avec scikit-learn, PyTorch ou Keras : c'est la méthode, pas l'outil, qui rend le résultat fiable.

Le modèle est un pipeline : vectoriser, puis classer

TP2

Un avis, texte brut

↓

TF-IDF : mots et paires de mots, 316 termes

↓

Régression logistique : un poids par terme

↓

positive ou negative, avec une probabilité

```
Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2),
                              min_df=2, sublinear_tf=True)),
    ("clf", LogisticRegression(max_iter=1000,
                               class_weight="balanced",
                               random_state=GRAINE)),
])
```

Un seul objet, sauvegardé d'un bloc : la même transformation à l'entraînement et en production.

TF-IDF : un terme pèse lourd s'il est fréquent dans l'avis et rare ailleurs

En détail TP2

```
poids(t, avis) = tf × idf

tf  = 1 + ln(occurrences de t dans l'avis)
idf = ln((1 + n) / (1 + df(t))) + 1

n     : nombre d'avis d'entraînement (232)
df(t) : nombre d'avis qui contiennent t

Chaque vecteur est ensuite normalisé :
sa longueur vaut 1.
```

Un terme présent partout

idf faible : il ne distingue aucun avis

Un terme présent dans peu d'avis

idf élevé : sa présence pèse lourd

Les réglages du TP2

sublinear_tf=True pour tf ; min_df=2 écarte les termes vus une seule fois ; ngram_range=(1, 2) ajoute les paires

La régression logistique additionne des poids, puis en fait une probabilité

En détail | TP2

```
z = b + somme des w(t) × poids(t)

p(positive) = 1 / (1 + e^(−z))

z > 0 : positive        z < 0 : negative
```

L'avis mitigé de la partie 2

z = −0,1343, donc p(positive) = 0,4665 : le modèle répond negative, avec une confiance de 0,5335.

Un poids par terme

positif s'il pousse vers « positive », négatif sinon

class_weight="balanced"

compense un déséquilibre des classes ; ici, 116 avis de chaque classe

Une décomposition exacte

z est une somme : SHAP la détaille terme par terme (complément du TP2)

On découpe avant de vectoriser, sinon le test fuit

TP2

290 avis de démonstration : 145 positifs, 145 négatifs

232 avis d'entraînement (80 %)	58 de test

Le bon ordre

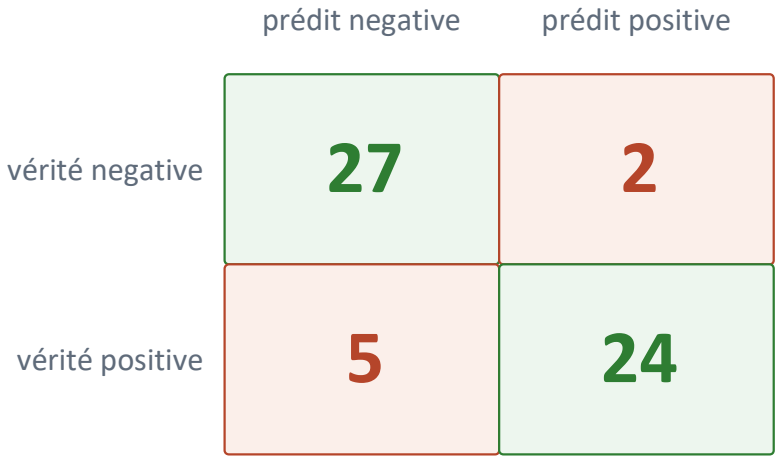
1. Découper : stratifié, graine 42
2. Apprendre le vocabulaire sur les 232 avis
3. Mesurer sur les 58 autres

Le piège

Vectoriser avant de découper : le vocabulaire du test entre dans l'entraînement, et le score devient trop beau.

F1 macro 0,879 : chaque classe compte autant

TP2



51 avis justes sur 58, 7 erreurs

Classe	Précision	Rappel	F1
negative	0,844	0,931	0,885
positive	0,923	0,828	0,873
F1 macro			**0,879**

Le rappel des négatifs, 0,931 : la part des mécontents que le modèle retrouve.

Précision, rappel et F1 se calculent à la main

En détail TP2

Classe negative

Précision = 27 / (27 + 5) = 0,844

Rappel = 27 / (27 + 2) = 0,931

F1 = 2 × 0,844 × 0,931 / (0,844 + 0,931) = 0,885

Classe positive

Précision = 24 / (24 + 2) = 0,923

Rappel = 24 / (24 + 5) = 0,828

F1 = 2 × 0,923 × 0,828 / (0,923 + 0,828) = 0,873

F1 macro = (0,885 + 0,873) / 2 = 0,879 **Exactitude = 51 / 58 = 0,879**

Pour une classe : VP = ses avis bien reconnus, FP = les avis classés à tort dans cette classe, FN = ses avis manqués.

La mesure entre dans Git, le modèle reste dehors

TP2

Dans Git

Le code, les tests, et models/metrics.json : la mesure de chaque entraînement

Hors de Git

models/sentiment.joblib et data/demo_reviews.csv : ils se reconstruisent en une commande

```
{
  "source": "demo",
  "n_train": 232, "n_test": 58,
  "f1_macro": 0.879,
  "recall_negative": 0.931
}
```

Les tests du TP2 lisent ce fichier : la CI vérifie le F1 sans avoir le modèle.

Les 7 erreurs sont des avis qui se retournent

TP2

« Taille trop petite et retour payant au déballage,

20 termes poussent vers negative

sauf que ensuite produit conforme à la description, très satisfait. »

13 termes poussent vers positive

Vérité : positive. Prédit : negative, confiance 0,5335.

sauf que ensuite | mais finalement | avant que | puis rapidement

À retenir : un modèle se mesure sur ce qu'il n'a jamais vu

1. Découper avant de vectoriser
2. Mesurer chaque classe : F1 macro et rappel
3. Garder la mesure dans Git, pas le modèle

Les contrôles du TP

C3
F1 0,879 ; metrics.json versionné

↓

C4
La matrice 27 2 / 5 24

↓

C5
42 passed, avec Allociné en plus

Partie 3, TP3

Exposer le modèle par une API

3

Le modèle est chargé une fois, puis chaque requête suit le même chemin

TP3

Au démarrage, lifespan charge le modèle une seule fois et le range dans app.state



Sans modèle, l'API refuse de démarrer : la panne se voit dès le déploiement (T20).

lifespan charge le modèle avant la première requête

En détail TP3

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Charge le modèle une seule fois, avant la première requête."""
    logger.info("Démarrage de l'API...")
    app.state.classifier = load_model()
    logger.info("Modèle chargé avec succès : %s", app.state.classifier.model_name)
    yield
    logger.info("Arrêt de l'API")

app = FastAPI(..., lifespan=lifespan)
```

Avant yield

exécuté une seule fois, au démarrage de l'API

app.state

le modèle, partagé par toutes les requêtes

Si load_model() échoue

uvicorn s'arrête : « Application startup failed »

La route /predict, ligne par ligne

En détail TP3

```
@app.post("/predict", response_model=PredictResponse, summary="Classifier un texte")
async def predict(request: PredictRequest):
    clf = _classifier()
    try:
        result = clf.predict(request.text)
    except ValueError as e:
        raise HTTPException(status_code=422, detail=str(e))
    except Exception:
        logger.exception("Erreur lors de la prédiction")
        raise HTTPException(status_code=500, detail="Erreur interne du serveur")
    logger.info("predict: %r -> %s (%.2f)", request.text[:50], result["label"], result["score"])
    return result
```

response_model

la réponse est vérifiée par le contrat avant d'être envoyée

ValueError : 422

toute autre erreur : 500

Le détail d'une panne

journalisé côté serveur, jamais renvoyé au client

Le contrat Pydantic refuse une requête invalide avant votre code

TP3

```
class PredictRequest(BaseModel):
    text: str = Field(..., min_length=1,
                      max_length=2000)

class PredictResponse(BaseModel):
    label: str
    score: float = Field(..., ge=0, le=1)
    text: str
```

{"text": ""}
422, string_too_short

{}
422, missing

{"text": 42}
422, string_type

Pydantic vérifie le corps avant que la fonction ne s'exécute : votre code ne voit que des requêtes valides.

Un refus 422 se lit champ par champ

En détail | TP3

```
{"detail": [
    {"type": "string_too_short",
     "loc": ["body", "text"],
     "msg": "String should have at least 1 character",
     "input": ""}
]}
```

detail est une liste : une entrée par champ fautif. Au TP3, trois espaces donnaient une simple chaîne, {"detail": "texte vide"} : le TP4 corrige cette différence.

type
la catégorie : string_too_short, missing, string_type

loc
où : dans le corps, le champ text

msg
le message lisible

input
la valeur reçue

Chaque code HTTP dit qui s'est trompé

TP3

Code	Signification	Exemple
200	Réussite : l'avis est classé	POSITIVE, confiance 0,777
422	Requête invalide : la faute du client	`{"text": ""}, {}, {"text": 42}`
503	Le modèle n'est pas chargé	Modèle retiré pendant que l'API tourne
500	Panne du serveur	Erreur inattendue dans predict()

Un 422 n'est jamais un 500 : l'erreur vient du client, et le serveur fonctionne.

Les réponses mesurées : POSITIVE 0,777 et NEGATIVE 0,7465

TP3

GET /health

```
{"status": "OK",
 "model": "tfidf-logreg (TP2, models/sentiment.joblib)"}
```

POST /predict
avis positif

```
{"label": "POSITIVE", "score": 0.777,
 "text": "Livraison rapide, produit conforme, très satisfait"}
```

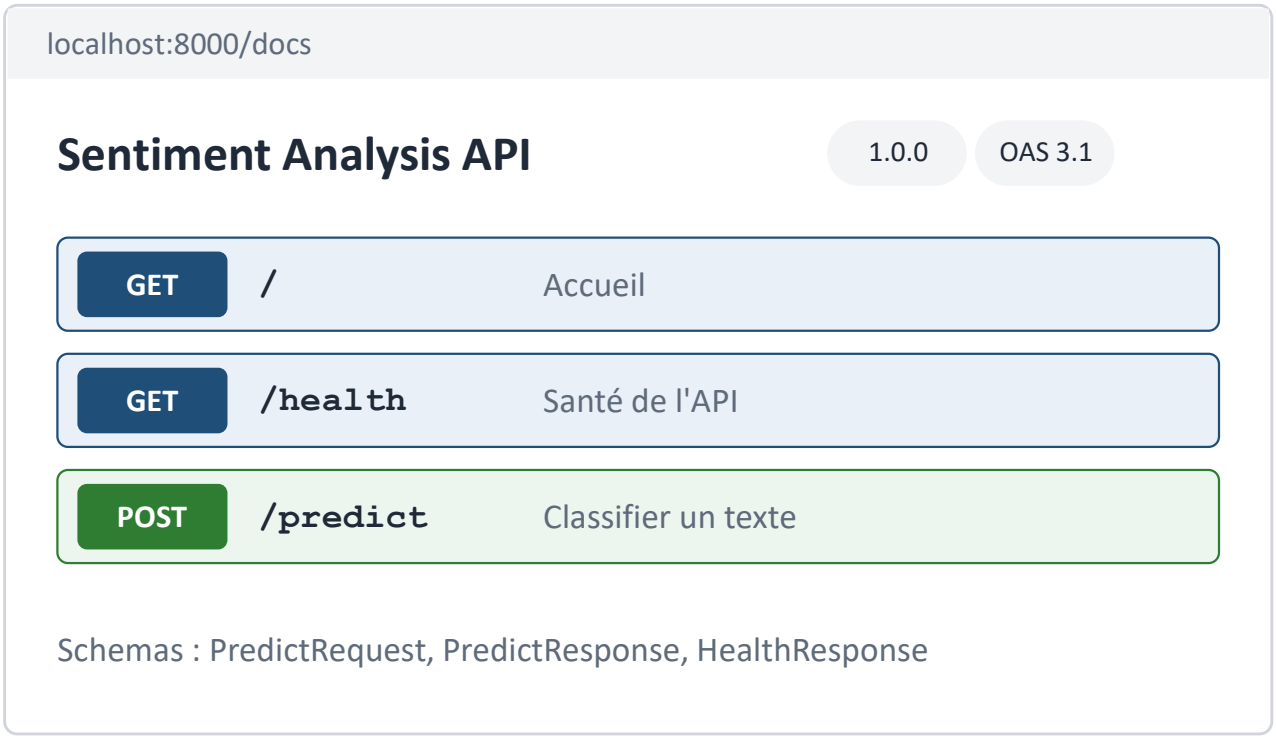
POST /predict
avis négatif

```
{"label": "NEGATIVE", "score": 0.7465,
 "text": "Produit cassé et service injoignable, à fuir"}
```

Scores exacts, parce que le modèle de démonstration est entraîné avec une graine fixée.

La page /docs est générée à partir du code

TP3



Construite à partir du code
routes, docstrings, contrats Pydantic

« Try it out », puis « Execute »
la requête, le code et le JSON reçus

503 et 500 n'y figurent pas
ils ne sont pas déclarés dans les routes

À retenir : une API est un contrat

1. Charger le modèle une fois ; sans lui, refuser de démarrer
2. Valider l'entrée avant le code
3. Un code HTTP par situation

Les contrôles du TP

T14
48 passed

↓

T15 à T18
200, puis 422 pour les trois requêtes invalides

↓

T20
Refus de démarrer sans modèle

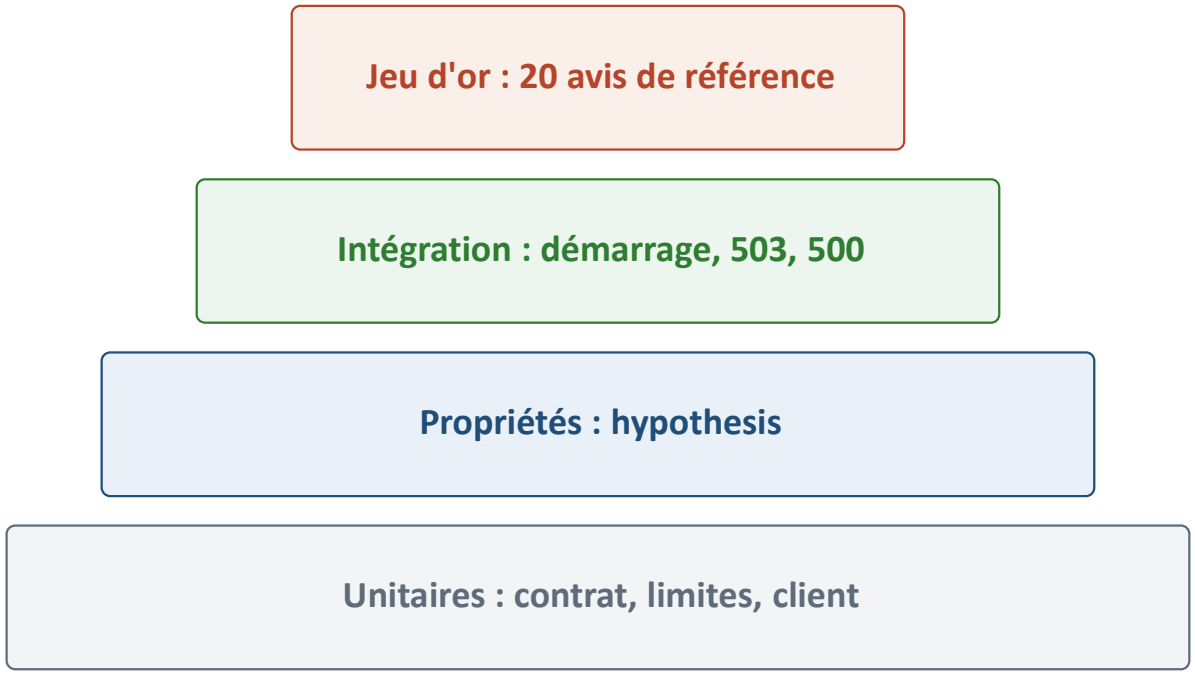
Partie 4, TP4

Tester et mesurer la qualité

4

Quatre sortes de tests, du plus rapide au plus réaliste

TP4



test_limites.py
8 cas limites et 2 propriétés

test_integration.py
6 tests

test_jeu_or.py
1 test d'ensemble, 1 par avis : 21

Une erreur connue se déclare, elle ne se cache pas

Un test d'intégration simule une panne du modèle

En détail TP4

```
class ModeleEnPanne:
    model_name = "modèle en panne (test)"
    def __init__(self, erreur: Exception) -> None:
        self.erreur = erreur
    def predict(self, text: str) -> dict:
        raise self.erreur

def test_modele_en_panne_repond_500(client_demarre, monkeypatch, erreur):
    from api.main import app
    monkeypatch.setattr(app.state, "classifier", ModeleEnPanne(erreur))
    reponse = client_demarre.post("/predict", json={"text": "Très bien"})
    assert reponse.status_code == 500
```

Un faux modèle

il lève l'erreur qu'on lui donne, à chaque prédiction

monkeypatch

remplace le modèle le temps d'un test, puis le remet

Ce que le test exige

une panne du modèle répond 500, jamais 422

Le jeu d'or déclare ses erreurs connues

En détail TP4

```
text,attendu,difficulte,erreur_connue
"Colis arrivé en avance, article parfait, je recommande",POSITIVE,,
"Emballage soigné, je suis très satisfait",POSITIVE,,
```

```
raison = (ligne.get("erreur_connue") or "").strip()
marques = [pytest.mark.xfail(strict=True, reason=raison)] if raison else []
```

20 avis de référence

11 justes, 8 abstentions, 1 erreur franche déclarée

xfail

une erreur déclarée est attendue : la suite reste verte

strict=True

le jour où l'erreur disparaît, le test le signale

Un test de propriétés trouve le cas qu'on n'imaginait pas

TP4

Avant

hypothesis réduit l'échec à "\x85", un blanc que str.strip() retire : le contrat le laissait passer.

Après

Le contrat applique la même règle que predict() : un seul refus, une seule forme de 422.

```
@field_validator("text")
@classmethod
def refuser_texte_blanc(cls, valeur: str) -> str:
    if not valeur.strip():
        raise ValueError("texte vide une fois les blancs retirés")
    return valeur
```

Deux propriétés, des centaines de textes générés

En détail TP4

```
@settings(max_examples=150, deadline=None)
@given(texte=st.text(min_size=1, max_size=2000).filter(lambda s: s.strip()))
def test_tout_texte_non_blanc_recoit_une_reponse(client, texte):
    reponse = client.post("/predict", json={"text": texte})
    assert reponse.status_code == 200
    assert reponse.json()["label"] in CONTRAT

@settings(max_examples=100, deadline=None)
@given(texte=st.text(alphabet=st.sampled_from(BLANCS), min_size=1, max_size=50))
def test_tout_texte_blanc_est_refuse(client, texte):
    reponse = client.post("/predict", json={"text": texte})
    assert reponse.status_code == 422
```

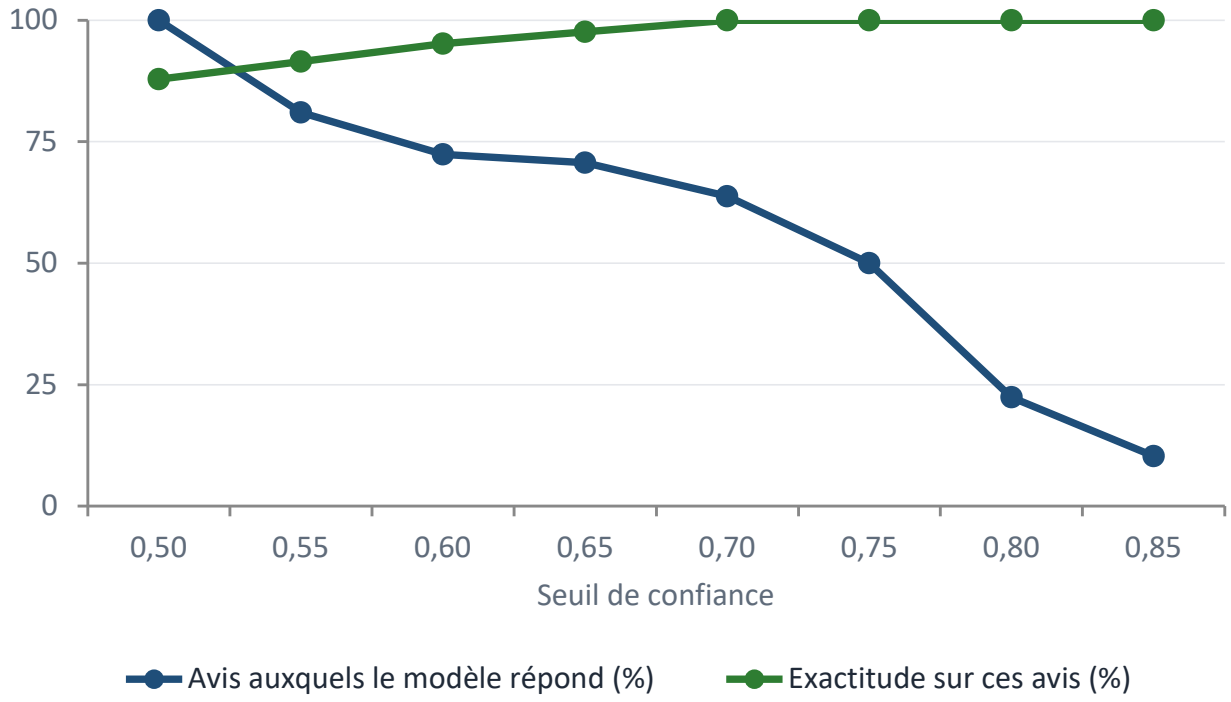
@given
décrit l'ensemble des entrées possibles, pas un exemple

En cas d'échec
hypothesis réduit l'entrée au plus petit contre-exemple

deadline=None
le premier appel, qui charge tout, peut être lent

Le seuil d'abstention échange de la couverture contre de la justesse

TP4



72 %

des avis traités au seuil 0,60

2 erreurs

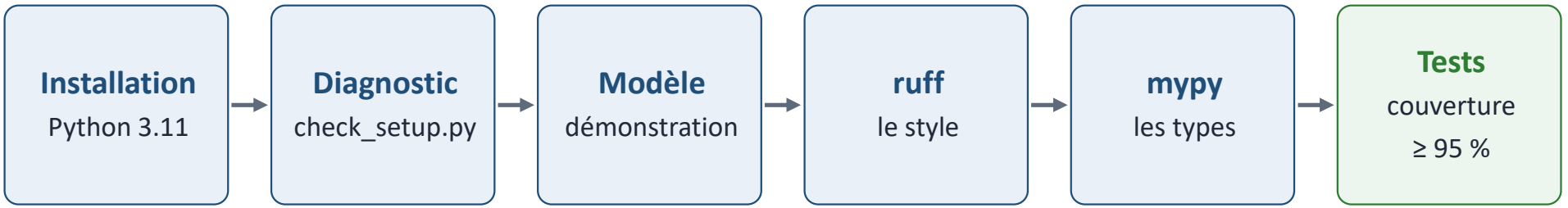
au lieu de 7

16 avis

confiés à un humain

La CI rejoue tout à chaque push

TP4



Couverture de l'API mesurée : 100 %

À chaque push, sur une machine neuve de GitHub : si une étape échoue, la croix rouge désigne le commit fautif, et le journal de l'étape nomme le test en cause.

La CI en YAML : une étape, une commande

En détail TP4

```
- name: Environnement virtuel et dépendances
  run: python -m venv .venv && . .venv/bin/activate && pip install ...
- name: Diagnostic
  run: . .venv/bin/activate && python scripts/check_setup.py
- name: Modèle de démonstration, pour les tests de l'API (TP3)
  run: . .venv/bin/activate && python src/make_demo.py && python src/train.py
- name: ruff
  run: . .venv/bin/activate && ruff check .
- name: mypy
  run: . .venv/bin/activate && mypy api
- name: Tests et couverture
  run: . .venv/bin/activate && pytest -q --cov=api --cov-report=term-missing --cov-fail-under=95
```

Une étape, un nouveau shell

chaque étape réactive l'environnement virtuel

Tout s'épingle

pytest installé sans version a suffi à casser une CI

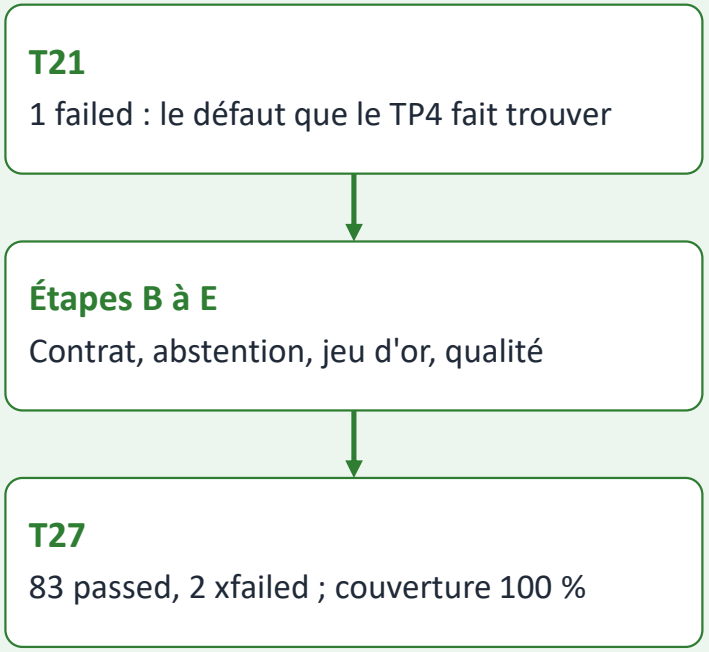
--cov-fail-under=95

sous 95 % de couverture, la CI échoue

À retenir : tester, c'est chercher ce qui casse

1. Quatre sortes de tests, du plus rapide au plus réaliste
2. Laisser hypothesis chercher les cas limites
3. Un seuil d'abstention se décide avec le métier

Les contrôles du TP



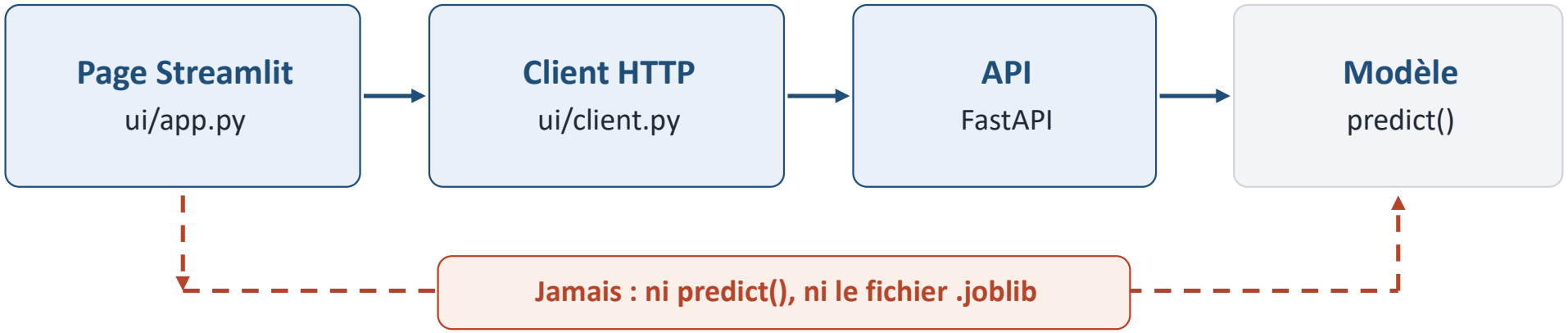
Partie 5, TP5

Donner une interface

5

La page est un client de l'API, comme un autre

TP5



L'adresse de l'API est dans API_URL : déplacer l'API ne change qu'une variable, et réentraîner le modèle ne change rien à la page.

Le client traduit chaque code en message

TP5

Réponse de l'API	Ce que la page affiche
200	Le verdict et la confiance
422	Texte refusé : vide, fait d'espaces, ou de plus de 2000 caractères.
503	Modèle non chargé : python src/train.py, puis relancer l'API.
Connexion impossible	API injoignable : lancez d'abord uvicorn api.main:app

Le client renvoie toujours une Reponse, jamais une exception : la page n'a qu'un seul cas à afficher.

Le client traduit les codes en une dizaine de lignes

En détail TP5

```
def predire(session: httpx.Client, texte: str) -> Reponse:
    try:
        r = session.post("/predict", json={"text": texte})
    except httpx.TransportError:
        return Reponse(False, message=INJOIGNABLE)
    if r.status_code == 200:
        corps = r.json()
        return Reponse(True, label=corps["label"], score=corps["score"])
    if r.status_code == 422:
        return Reponse(False, message=REFUS)
    if r.status_code == 503:
        return Reponse(False, message=SANS_MODELE)
```

session

un client httpx ; dans les tests, un TestClient le remplace

TransportError

la connexion est impossible : l'API n'est pas lancée

Toujours une Reponse

ok, verdict, confiance ou message : la page n'a qu'un cas

Deux onglets : un avis, ou un fichier entier

TP5

Un avis | Un fichier d'avis

Livraison rapide, produit conforme, très satisfait

Analyser

Avis positif
78%

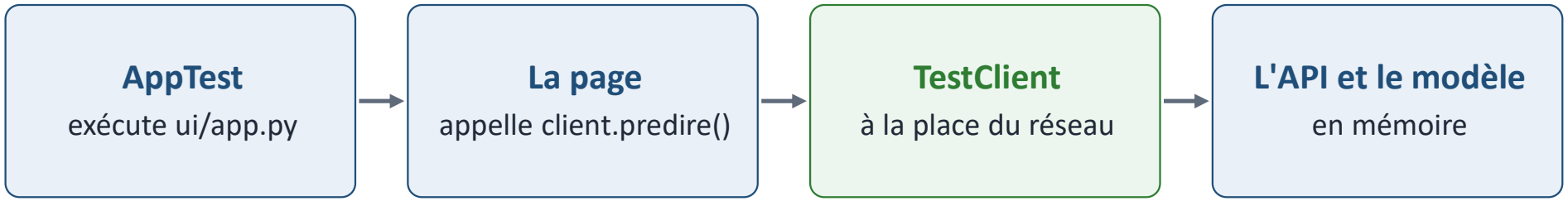
data/avis_exemple.csv : 6 avis

3	2	1
positifs	négatifs	à vérifier

Le tableau des avis, un résumé et deux graphiques

Les tests exécutent la vraie page, sans serveur

TP5



test_client.py : 7 tests
le client et ses messages

test_interface.py : 5 tests
la vraie page, saisie et affichage

Aucun serveur à lancer : les mêmes tests tournent dans la CI.

Un test de la page : saisir, cliquer, lire ce qui s'affiche

En détail TP5

```
@pytest.fixture
def page(api, monkeypatch):
    monkeypatch.setattr(client, "creer_session", lambda: api)  # la page parle à l'API en mémoire
    return AppTest.from_file(PAGE, default_timeout=30).run()

def analyser(page, texte):
    page.text_area[0].input(texte).run()
    page.button[0].click().run()
    return page

def test_avis_positif(page):
    assert analyser(page, "Livraison rapide, ...").metric[0].label == "Avis positif"
```

monkeypatch
la page reçoit une API en mémoire au lieu du réseau

AppTest
exécute la vraie page, sans navigateur

Ce qu'on vérifie
ce que l'utilisateur lirait : le libellé « Avis positif »

À retenir : l'interface ne sait rien du modèle

1. La page ne parle qu'à l'API
2. Chaque code devient un message clair
3. Tester la vraie page, sans serveur

Les contrôles du TP

T28 à T31
Kit installé, API et page lancées

↓

T32, T33
Erreurs traduites ; fichier d'avis

↓

T34
95 passed, 2 xfailed

Partie 6, TP6

Livrer un paquet installable

6

Un paquet contient le code, pas les tests, les données ni le modèle

TP6

Dans la roue

- api/
- sentiment_app/
- ui/
- métadonnées : version, dépendances, commande

Hors de la roue

- tests/
- data/
- models/ : le modèle, indiqué par SENTIMENT_MODELE
- src/

sentiment_app-0.6.0-py3-none-any.whl : le modèle a son propre cycle de vie ; on le réentraîne sans republier le code.

pyproject.toml décrit tout le paquet

TP6

```
[project]
dynamic = ["version"]
dependencies = [
    "fastapi>=0.115,<0.116",
    "scikit-learn>=1.5,<1.6",
]
[project.scripts]
sentiment = "sentiment_app.cli:main"
[tool.setuptools]
packages = ["sentiment_app", "api", "ui"]
[tool.setuptools.dynamic]
version = { attr = "sentiment_app.__version__" }
```

Des dépendances épinglées

pip installe les versions compatibles avec le modèle

Une commande, sentiment

créée par pip à l'installation

Une seule source pour la version

sentiment_app.__version__ : 0.6.0

La commande sentiment marche partout, si on lui indique le modèle

TP6

```
> sentiment --version
sentiment-app 0.6.0
> sentiment predict "Livraison rapide, produit conforme, très satisfait"
{"label": "POSITIVE", "score": 0.777, ...}
> sentiment predict "xqzvb wlkptr grombu"
{"label": "INDETERMINE", "score": 0.5162, ...}
```

Hors du dépôt, sans la variable

« modèle introuvable », code de sortie 1

Avec SENTIMENT_MODELE

POSITIVE, 0,777, depuis n'importe quel dossier

La commande sentiment : argparse et trois sous-commandes

En détail TP6

```
def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="sentiment", description="...")
    parser.add_argument("--version", action="version", version=f"sentiment-app {__version__}")
    commandes = parser.add_subparsers(dest="commande", required=True)
    predire = commandes.add_parser("predict", help="classe un avis, sans serveur")
    predire.add_argument("texte", nargs="+", help="l'avis à analyser")
    predire.set_defaults(fonction=_predire)
    servir = commandes.add_parser("serve", help="lance l'API")
    servir.set_defaults(fonction=_servir)
    interface = commandes.add_parser("ui", help="lance la page Streamlit (l'API doit tourner)")
    args = parser.parse_args(argv)
    return int(args.fonction(args))
```

--version

lit __version__ : la version n'est écrite qu'une fois

predict

passe par api/model.py : le même seuil d'abstention que l'API

[project.scripts]

relie la commande sentiment à cette fonction main()

Construire, installer, essayer : la roue de bout en bout

En détail TP6

```
> python -m build --wheel
Successfully built sentiment_app-0.6.0-py3-none-any.whl
> py -3.11 -m venv $env:TEMP\roue-tp6
> $env:TEMP\roue-tp6\Scripts\pip install dist\sentiment_app-0.6.0-py3-none-any.whl
> $env:TEMP\roue-tp6\Scripts\sentiment predict "Livraison rapide"
modèle introuvable : ... ou indiquez son chemin dans la variable SENTIMENT_MODELE
> $env:SENTIMENT_MODELE = "C:\IA-DEV\sentiment-app\models\sentiment.joblib"
> $env:TEMP\roue-tp6\Scripts\sentiment predict "Livraison rapide, produit conforme, très satisfait"
{"label": "POSITIVE", "score": 0.777, ...}
```

Un environnement neuf

la roue s'y installe avec ses dépendances, sans le dépôt

Le modèle à part

désigné par la variable SENTIMENT_MODELE

La CI le rejoue

à chaque push, avec la même suite de commandes (T41)

Un modèle ne se relit qu'avec la version qui l'a produit

TP6

Entraîné par la CI
scikit-learn 1.9.1, sans version imposée

→

Relu par la roue
scikit-learn 1.5.2

→

Échec au chargement

```
AttributeError: 'LogisticRegression' object has no attribute 'multi_class'
```

La correction
scikit-learn>=1.5,<1.6 dans pyproject.toml, scikit-learn==1.5.* dans la CI : le modèle est entraîné et relu avec la même version.

À retenir : livrer, c'est rendre reproductible

1. Le paquet contient le code, pas le modèle
2. Une seule source pour la version : 0.6.0
3. Épingler ce qui lit le modèle

Les contrôles du TP

T35
2 failed voulus au départ

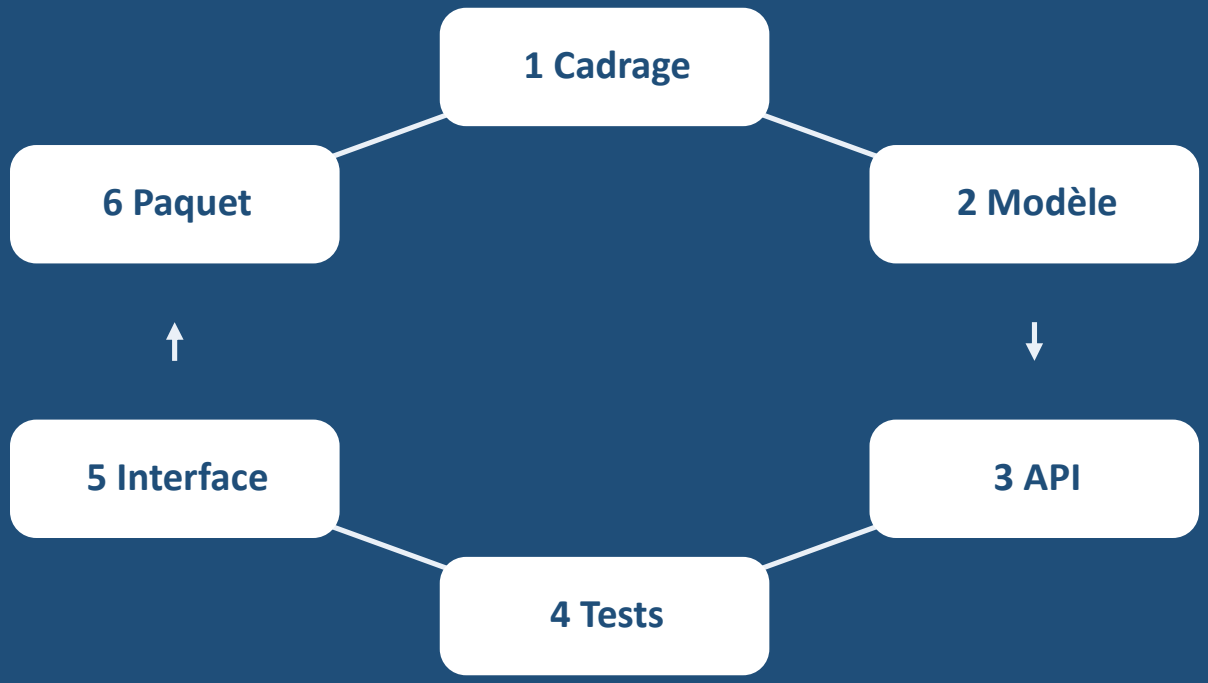
↓

T39, T40
La roue construite, puis installée ailleurs

↓

T41
101 passed, 2 xfailed ; v0.6.0 ; CI verte

Six étapes, un cycle, et la suite



La suite

TP7
MLflow et DVC : suivre les expériences et les données

TP8
Docker : conteneuriser et déployer

TP9
RAG : répondre en citant ses sources

Annexe : les commandes essentielles

Annexe

TP	Commandes, environnement activé	Résultat attendu
TP1	`py -3.11 -m venv .venv ; .venv\Scripts\Activate.ps1 ; pip install -r requirements.txt`	38 passed en fin de TP1
TP2	`python src/make_demo.py ; python src/train.py`	F1 macro 0,879
TP3	`uvicorn api.main:app --reload`	La page /docs, port 8000
TP4	`python -m pytest -q --cov=api ; ruff check . ; mypy api`	Couverture de 100 %
TP5	`streamlit run ui/app.py`	La page, API lancée
TP6	`python -m build --wheel ; sentiment --version`	sentiment-app 0.6.0